JAIN DEEMED-TO-BE UNIVERSITY | FACULTY OF ENGINEERING AND TECHNOLOGY

School of Computer Science and Engineering

PYTHON PROGRAMMING

MINI PROJECT REPORT

RETAIL SALES DASHBOARD USING CSV AND MATPLOTLIB

A Python-based system for retail transaction analysis, business metrics and data visualization

Name	HAIKRIVA R B
USN	25BTRPW043
Class / Section	CSE-PWC(A)
Subject Name	Python Programming
Faculty Name	Ms. Sujithra G
Date of Report Submission	02/11/2026
Project Topic	Retail Sales Dashboard Using CSV and Matplotlib
Project Components	CSV Processing, Matplotlib and Flask

TABLE OF CONTENTS

S. No.	Topic	Page No.
1	Introduction	3
2	Problem Statement	3
3	Objectives	3
4	Project Description	4
5	Concepts Used in the Program	4–5
6	Algorithm	5
7	Flowchart	6
8	Python Program – Core Implementation	7–9
9	Output and Dashboard	10
10	Conclusion	11

1. INTRODUCTION

Retail businesses generate transaction data whenever customers purchase products. Each transaction may contain details such as date, product, category, quantity, unit price, discount, cost, payment method, city, order status and customer rating. Studying these records helps a business understand its sales performance and customer behaviour.

The **Retail Sales Dashboard Using CSV and Matplotlib** is a Python mini project that converts transaction records into useful business information. The project uses a CSV file as its data source. Python reads and processes the records, calculates key performance indicators (KPIs), groups sales information and creates visual charts using Matplotlib.

The project also provides a command-line menu and a browser-based dashboard developed with Flask. Users can view an executive summary, filter records by category or city, generate a Matplotlib dashboard, export a text report, regenerate sample data and explore the interactive dashboard.

The uploaded dataset contains 1,201 retail transactions dated during 2025 and covers six product categories and eight cities. The figures in this report are calculated from the supplied CSV file and the category configuration currently included in the project.

2. PROBLEM STATEMENT

Retail transaction data stored in CSV files can be difficult to interpret when it is viewed row by row. Manual calculations of revenue, profit, product performance and customer patterns take time and may lead to mistakes. A simple system is therefore required to read the CSV data, process it automatically, summarize important measures and display the results in charts.

The problem is to design and develop a Python-based Retail Sales Dashboard that loads retail transaction records from a CSV file, calculates sales and profitability indicators, supports category and city analysis, and presents the results through Matplotlib visualizations and an interactive web interface.

3. OBJECTIVES

- To read retail transaction records from a CSV file using Python.
- To calculate total revenue, net profit, profit margin, average order value and other KPIs.
- To analyse sales by month, product category, city, payment method and day of the week.
- To identify top-performing products and customer segments.
- To create a multi-chart sales dashboard using Matplotlib.
- To provide filtering, report export and dataset regeneration options.
- To provide an interactive browser dashboard using the Flask framework.

4. PROJECT DESCRIPTION

The project is organized into separate Python modules. This modular structure keeps data generation, data analysis, chart creation and user interaction in different files. The main program acts as the controller and connects the modules.

4.1 Data Source and Loading

The file **retail_sales_data.csv** stores the transaction records. The dataset includes transaction ID, date, time, customer ID, gender, age group, city, product category, product name, quantity, unit price, unit cost, discount percentage, total amount, cost total, net profit, payment method, order status and rating. The analyzer reads the file with **csv.DictReader**, which makes each row accessible through its column headings.

4.2 Data Processing and Analysis

The RetailSalesAnalyzer class loads the CSV records and prepares values such as dates, quantities and monetary amounts. It calculates KPIs and groups records using dictionaries and defaultdict. It can filter records by category, city, date range or quarter. Category configuration is stored in a JSON file and can adjust selling price, cost, discount, target revenue, category name and active status.

4.3 Dashboard Generation

The dashboard.py module uses Matplotlib to generate a large dashboard image. GridSpec arranges a KPI banner and six chart areas. The charts show monthly revenue and profit, category revenue and profit, payment method share, customer RFM segments, city performance and sales by day of the week. The generated image is saved as **retail_sales_dashboard.png**.

4.4 Interactive Web Application

The app.py module uses Flask to serve the browser interface. It provides routes for dashboard data, category configuration, simulated sales, report export, CSV export and CSV upload. The HTML interface is stored in templates/index.html. The browser dashboard uses the API data to update its visualizations and transaction table.

4.5 Main Project Files

File	Purpose
main.py	Command-line menu and application control
sales_analyzer.py	CSV loading, KPI calculations, filtering and analysis
dashboard.py	Matplotlib dashboard creation
generate_data.py	Generation of sample retail transaction records
app.py	Flask web application and API routes
category_config.json	Editable category settings
retail_sales_data.csv	Retail transaction dataset
templates/index.html	Browser dashboard interface

5. CONCEPTS USED IN THE PROGRAM

CSV File Handling: The csv module reads and writes comma-separated transaction data. csv.DictReader reads the header row and maps each record to a dictionary, which reduces the need to access fields by numeric column positions.

Functions: Functions divide the program into reusable tasks. Examples include create_dashboard(), generate_retail_sales_csv(), get_kpis(), get_monthly_sales() and generate_text_summary().

Classes and Objects: The RetailSalesAnalyzer class groups related data and analysis methods. An object of this class is created using the CSV file path and is used by the command-line and web application.

Lists and Dictionaries: Lists store collections of transaction records and chart labels. Dictionaries store each transaction, category configuration and grouped analysis results.

Loops and Conditional Statements: Loops process transaction rows and display menu options. if/elif/else statements select menu actions, validate choices and classify customer segments.

Exception Handling: try/except blocks handle invalid or incomplete transaction values while loading CSV records and protect selected file and web operations from terminating unexpectedly.

Matplotlib: Matplotlib creates line charts, bar charts, horizontal bar charts and a doughnut chart. These visualizations make changes and comparisons easier to understand.

GridSpec Layout: matplotlib.gridspec.GridSpec arranges the dashboard into sections with different sizes. This allows the KPI banner to use the full width while six charts are placed below it.

Flask and JSON: Flask provides web routes and returns analysis results as JSON. JSON is also used to save and load editable category settings.

Data Aggregation: collections.defaultdict is used to accumulate revenue, profit, quantities and counts by month, category, city, product and payment method. This supports efficient one-pass grouping.

5.1 Important Calculations

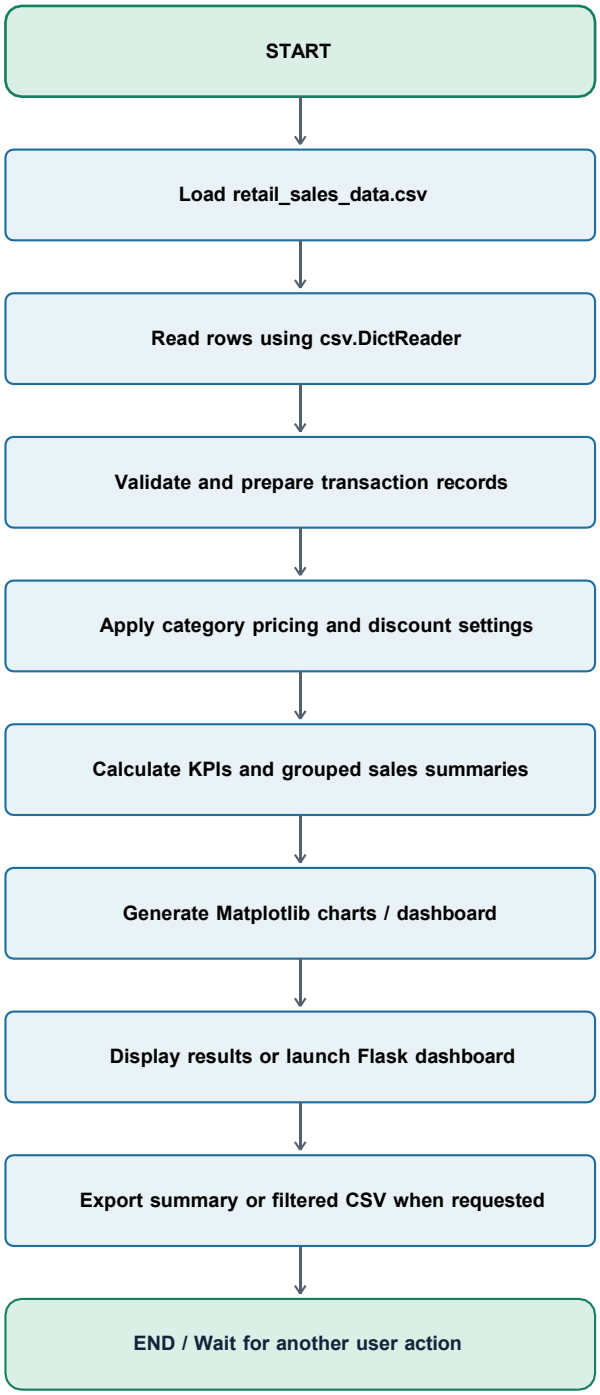
- **Total Revenue** = Sum of the transaction Total_Amount values.
- **Net Profit** = Total Revenue – Total Cost.
- **Profit Margin (%)** = (Net Profit ÷ Total Revenue) × 100.
- **Average Order Value** = Total Revenue ÷ Number of Transactions.
- **Month-over-Month Growth (%)** = ((Current Month Revenue – Previous Month Revenue) ÷ Previous Month Revenue)
  × 100.
- **Return Rate (%)** = (Number of Returned Transactions ÷ Total Transactions) × 100.

6. ALGORITHM

1. Start the application.
2. Check whether the retail CSV dataset exists. If it is missing, generate a sample dataset.
3. Create a RetailSalesAnalyzer object and load the CSV records.
4. Load the category configuration from the JSON file and apply the configured values.
5. Display the main menu and ask the user to select an operation.
6. If the user selects the executive summary, calculate and display the KPIs and analysis.
7. If the user selects the Matplotlib dashboard, collect the grouped data and draw the charts.
8. If the user selects filtering, apply the selected category and city and calculate filtered KPIs.
9. If the user selects report export, write the text summary to a report file.
10. If the user selects dataset regeneration, create new CSV records and reload the analyzer.

7. FLOWCHART

The following flowchart shows the main processing sequence of the Retail Sales Dashboard. The application loads the CSV data, prepares the records, performs analysis and then displays or exports the requested results.



Note: The command-line menu and Flask web interface provide different ways to access the same analysis engine. The flowchart represents the overall project workflow rather than every individual web route.

8. PYTHON PROGRAM

The complete project is divided into multiple source files. The following selected sections show the central implementation logic used for CSV loading, data preparation and KPI calculations. The original source files are retained in the uploaded project ZIP.

```python
"""
main.py
Interactive CLI controller for the Retail Sales Dashboard Mini Project.
"""

import sys
import os
from datetime import datetime
from sales_analyzer import RetailSalesAnalyzer
from dashboard import create_dashboard
from generate_data import generate_retail_sales_csv

DEFAULT_CSV = "retail_sales_data.csv"

def print_header():
    print("\n" + "=" * 65)
    print("      RETAIL SALES ANALYTICS & DASHBOARD SYSTEM")
    print("      Mini Project: CSV Data Processing + Matplotlib")
    print("=" * 65)

def print_menu():
    print("\n[MENU OPTIONS]")
    print("  1. View Executive Summary & Key KPIs (Terminal)")
    print("  2. Generate & View Matplotlib Visual Dashboard")
    print("  3. Filter Analysis by Category or City")
    print("  4. Export Summary Report to File")
    print("  5. Regenerate Retail Dataset (Custom Record Count)")
    print("  6. Launch Interactive Web Dashboard (Flask on http://127.0.0.1:5000)")
    print("  0. Exit")
    print("-" * 65)

def export_report_to_file(analyzer, output_file="sales_summary_report.txt"):
    report = analyzer.generate_text_summary()
    with open(output_file, "w", encoding="utf-8") as f:
        f.write(report)
        f.write(f"\nReport generated on: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n")
    print(f"\n[SUCCESS] Report exported to '{output_file}'.")

def filter_menu(analyzer):
```

```python
def filter_menu(analyzer):
    print("\n--- Filter Analysis ---")
    print("Available Categories:")
    cats = analyzer.get_category_sales()["categories"]
    for i, c in enumerate(cats, 1):
        print(f"  {i}. {c}")
    print("  0. All Categories")

    cat_choice = input("Select Category (0-6): ").strip()
    selected_cat = None
    if cat_choice.isdigit() and 1 <= int(cat_choice) <= len(cats):
        selected_cat = cats[int(cat_choice) - 1]

    print("\nAvailable Cities:")
    cities = analyzer.get_city_sales()["cities"]
    for i, city in enumerate(cities, 1):
        print(f"  {i}. {city}")
    print("  0. All Cities")

    city_choice = input("Select City (0-8): ").strip()
    selected_city = None
    if city_choice.isdigit() and 1 <= int(city_choice) <= len(cities):
        selected_city = cities[int(city_choice) - 1]

    filtered_records = analyzer.filter_records(category=selected_cat, city=selected_city)
    kpis = analyzer.get_kpis(filtered_records)

    print("\n" + "=" * 55)
    print(f" FILTERED RESULTS: Category='{selected_cat or 'All'}', City='{selected_city or 'All'}'")
    print("=" * 55)
    print(f"  Matching Transactions: {kpis['total_transactions']:,}")
    print(f"  Filtered Revenue:      ${kpis['total_revenue']:,.2f}")
    print(f"  Total Units Sold:      {kpis['total_units']:,}")
    print(f"  Avg Order Value:       ${kpis['avg_order_value']:,.2f}")
    print(f"  Unique Customers:      {kpis['unique_customers']:,}")
    print("=" * 55)

def main():
    csv_file = DEFAULT_CSV
```

```
Click to add a breakpoint
77      csv_file = DEFAULT_CSV
78      if not os.path.exists(csv_file):
79          print(f"[INFO] '{csv_file}' not found. Generating default dataset...")
80          generate_retail_sales_csv(csv_file, num_records=1000)
81
82      analyzer = RetailSalesAnalyzer(csv_file)
83
84      while True:
85          print_header()
86          print(f"  Dataset: {csv_file} ({len(analyzer.records):,} loaded records)")
87          print_menu()
88          choice = input("Enter your choice (0-6): ").strip()
89
90          if choice == "1":
91              print("\n" + analyzer.generate_text_summary())
92              input("\nPress Enter to continue...")
93
94          elif choice == "2":
95              print("\n[INFO] Generating Matplotlib Dashboard...")
96              create_dashboard(csv_path=csv_file, output_image="retail_sales_dashboard.png", show_plot=True)
97              input("\nPress Enter to continue...")
98
99          elif choice == "3":
100             filter_menu(analyzer)
101             input("\nPress Enter to continue...")
102
103         elif choice == "4":
104             export_report_to_file(analyzer)
105             input("\nPress Enter to continue...")
106
107         elif choice == "5":
108             count_str = input("\nEnter number of records to generate (e.g. 500, 2000): ").strip()
109             count = int(count_str) if count_str.isdigit() and int(count_str) > 0 else 1000
110             generate_retail_sales_csv(csv_file, num_records=count)
111             analyzer = RetailSalesAnalyzer(csv_file)
112             print(f"[SUCCESS] Reloaded dataset with {len(analyzer.records):,} records.")
113             input("\nPress Enter to continue...")
114
```

```
76  def main():
84      while True:
99          elif choice ==  3 :
102
103         elif choice == "4":
104             export_report_to_file(analyzer)
105             input("\nPress Enter to continue...")
106
107         elif choice == "5":
108             count_str = input("\nEnter number of records to generate (e.g. 500, 2000): ").strip()
109             count = int(count_str) if count_str.isdigit() and int(count_str) > 0 else 1000
110             generate_retail_sales_csv(csv_file, num_records=count)
111             analyzer = RetailSalesAnalyzer(csv_file)
112             print(f"[SUCCESS] Reloaded dataset with {len(analyzer.records):,} records.")
113             input("\nPress Enter to continue...")
114
115         elif choice == "6":
116             import subprocess
117             import webbrowser
118             print("\n[INFO] Launching Flask Web Dashboard...")
119             print("Opening browser at http://127.0.0.1:5000 (Press Ctrl+C in server window to stop)...")
120             try:
121                 webbrowser.open("http://127.0.0.1:5000")
122                 from app import app
123                 app.run(debug=False, host="127.0.0.1", port=5000)
124             except KeyboardInterrupt:
125                 print("\n[INFO] Web dashboard stopped.")
126             input("\nPress Enter to continue...")
127
128         elif choice == "0":
129             print("\nExiting Retail Sales Dashboard. Thank you!\n")
130             sys.exit(0)
131
132         else:
133             print("\n[ERROR] Invalid choice. Please choose 0 to 6.")
134
135 if __name__ == "__main__":
136     main()
137
```

9. OUTPUT AND DASHBOARD

The dashboard image below is the actual Matplotlib output generated from the uploaded project dataset and current category configuration. It contains four KPI cards and six visualizations.



9.1 Sample Executive Results

Metric	Result from supplied dataset
Total revenue	$142,205.66
Net profit	$60,270.64
Profit margin	42.4%
Transactions	1,201
Units sold	2,087
Average order value	$118.41
Unique customers	340
Average customer rating	4.64 / 5.00
Return rate	12.7%
Top category / product	Electronics / Smartphone

The supplied CSV contains 1,201 transaction rows. The totals above are recalculated from the CSV using the current category configuration. The project can produce different values if the dataset is regenerated, uploaded or category settings are changed.

10. CONCLUSION

The Retail Sales Dashboard Using CSV and Matplotlib project successfully demonstrates how Python can be used to process retail transaction data and convert it into meaningful business information. The system reads data from a CSV file, prepares the records and calculates important measures such as revenue, net profit, profit margin, average order value, return rate and customer rating.

The project analyses sales from different perspectives, including month, product category, city, payment method, product, customer segment and day of the week. Matplotlib is used to present these results through charts in a consolidated dashboard. The use of a GridSpec layout helps organize the KPI cards and multiple visualizations in a single figure.

The project also demonstrates modular programming through separate files for data generation, analysis, dashboard creation and application control. The command-line interface provides menu-based access to the functions, while the Flask web application adds browser-based interaction, filtering, CSV upload and export, report download, category configuration and live sale simulation.

The supplied dataset contains 1,201 transactions, with calculated revenue of $142,205.66 and net profit of $60,270.64 under the current configuration. These results illustrate how the application can summarize a large set of transaction records and make comparisons easier through visual representation.

Overall, this mini project combines CSV file handling, Python data structures, functions, classes, conditional statements, loops, exception handling, Matplotlib and Flask in a practical application. In the future, the project can be extended with a database, user authentication, real point-of-sale data, scheduled reports and more advanced forecasting methods.

PROJECT DETAILS

Project Topic: Retail Sales Dashboard Using CSV and Matplotlib
Student: HAIKRIVA R B
USN: 25BTRPW043
Class / Section: CSE-PWC(A)
Subject: Python Programming
Faculty: Ms. Sujithra G
Submission Date: 02/11/2026

— END OF REPORT —